

3. Voraussetzungen

4. Quelltext

5. Pakete, Klassen, Variablen

6. Methoden

- Kapselung und Konstruktoren
- Methoden
- Klassen- vs. Instanzmethoden
- Geteilter und exklusiver Zugriff
- Bindung
- Methodenausführung

Kapselung und Konstruktoren

Kapselung

Zustand eines Objektes nicht unbedingt für alle möglichen Werte von Instanzvariablen gültig

z.B. **Einschränkungen** oder **Zusammenhänge** von Werten:

- Alter liegt im Bereich $[0; 127]$
- Ehepartner/in erst ab 18 möglich
- ...

um **gültigen** Zustand zu garantieren: **Variablen private deklarieren!**
⇒ **Zugriff** auf und **Manipulation** von Variablen **nur durch Methoden**

Effekt der **Kapselung**: jedes **Objekt kontrolliert seinen Zustand**
⇒ Anpassung der **Implementierung auf Klasse beschränkt**

Initialisierung

Zustand eines Objektes muss **bei Instanziierung initialisiert** werden, damit er von Anfang an **wohldefiniert** ist (**sonst Fehler** möglich!):

implizite Init. von Variablen mit 0, 0.0, false, null

explizite Init. mit **allgemeinem Wert** möglich:

```
private double t = 37.0; // Koerpertemperatur
```

Frage: wie **Init.** mit **individuellem Wert** (z.B. für Name von Mensch)?

Initialisierung (Vorschlag): per Instanzmethode

```
public class Mensch {  
    // ...  
    public void init(String name, int jahr) {  
        ++a;  
        n = name;  
        j = jahr;  
    }  
}
```

Problem: Aufruf von Initialisierungsmethode `init` eventuell

- **ausgelassen** $\Rightarrow$ ggf. **ungültiger Zustand**
- **wiederholt** $\Rightarrow$ ggf. **unsinnig** (a um 2 erhöht)
 ggf. durch **Unbefugte** (nicht „Schöpfer“ des Objekts)

Initialisierung bei Instanziierung: Konstruktoren

Lösung: Initialisierung (nur) bei Instanziierung

konkreter: Erzeugen einer Instanz durch `new` mit Argumenten:

```
Mensch mo = new Mensch("Monika", 1992);  
Mensch ma = new Mensch("Markus", 1998);
```

Konstruktoren: spezielle „Instanzmethoden“, die

- genau einmal pro Instanz aufgerufen werden: bei Instanziierung
- nur dazu dienen, Instanzvariablen individuell zu initialisieren (und ggf. Klassenvariablen zu aktualisieren)

```
public Mensch(String name, int jahr) {  
    ++a;  
    n = name;  
    j = jahr;  
}
```

Konstruktor: Deklaration

KonstruktorDeklaration:

```
Zugriff KlassenBez ( ParamListe ) { { Anweisung } }
```

ParamListe:

```
[ Param { , Param } ]
```

Param:

```
Typ ParamBez
```

syntaktische Unterschiede gegenüber Instanzmethoden:

- kein eigener Name, sondern Name der Klasse
- kein Rückgabetyt bzw. -wert ($\sim$ void-Methode)
- kein direkter, bel. häufiger Aufruf, sondern genau einmal per new

this

in **Konstruktoren** und generell in **Instanzelementen**:

this ist **Referenz** auf **betrachtete Instanz**, modelliert „ich“/„mein“

in Instanzelementen wird **unqualifizierter Zugriff**

```
public Mensch(String name, int jahr) {  
    ++a;  
    n = name;  
    j = jahr;  
}
```

übersetzt in **qualifizierten Zugriff**

```
++Mensch.a;  
this.n = name;  
this.j = jahr;
```

mittels **Klassenname** (**Klassenelement**) bzw. **this** (**Instanzelement**)

Auflösen von Namenskonflikten

gleiche Namen für Instanz-/Klassen- und lokale Variablen/Parameter lassen sich mittels `this` unterscheiden:

```
6 | public Mensch(String n, int j) {  
7 |     ++a;  
8 |     this.n = n;  
9 |     this.j = j;  
10| }
```

mit `this` expliziter Bezug auf Instanz-/Klassenvariable, sonst lokale
→ Erfinden verschiedener Namen für Gleiches nicht erforderlich

Style Guidelines:

- selbe Namen nur für Gleiches verwenden
- Qualifikation mit `this` nur bei Namenskonflikt verwenden (sonst irreführend, da Verwendung Konflikt suggeriert)

Überladen von Konstruktoren: Beispiel Punkt2D

Beispiel: geometrischer Punkt im 2-dimensionalen Raum

```
1 public class Punkt2D {
2     private static int a;
3     private double x; // x-Koordinate
4     private double y; // y-Koordinate
5     public Punkt2D() {
6         ++a;
7     }
8     public Punkt2D(double x, double y) {
9         ++a;
10        this.x = x;
11        this.y = y;
12    }
    public Punkt2D(Punkt2D p) {
        ++a;
        x = p.x;
        y = p.y;
    }
}
```

Überladen von Konstruktoren: Wirkung

Instanziierung hier auf 3 Weisen durch **überladene** Konstruktoren:

- Default-Instanziierung ohne Argumente
- initialisierende Instanziierung mit Werten für Elemente
- Kopie-Instanziierung nach Vorlage

Voraussetzung: untersch. Anzahl/Typen der Parameter

```
Punkt2D p1 = new Punkt2D();      // erzeugt (0,0)
Punkt2D p2 = new Punkt2D(8,3);   // erzeugt (8,3)
Punkt2D p3 = new Punkt2D(p2);    // erzeugt (8,3)
```

Default-Konstruktor

(nur) wenn für eine Klasse kein Konstruktor deklariert wird, wird automatisch der **Default-Konstruktor** (ohne Argumente) erzeugt

→ Körper des Konstruktors ist dabei leer

sobald Konstruktoren deklariert werden, wird Default-Konstruktor nicht erzeugt, sondern muss bei Bedarf explizit deklariert werden

→ Körper des Konstruktors kann dann Anweisungen enthalten

Delegation von Konstruktoraufruf

Konstruktor kann (als erste Anweisung) **anderen Konstruktor aufrufen** und so die **Initialisierung delegieren**:

KonstruktorAufrufAnweisung:

```
this ( [ Ausdruck { , Ausdruck } ] ) ;
```

danach sind weitere Anweisungen möglich

Beispiel: Ersetzung des zuvor gezeigten Kopie-Konstruktors

```
13 | public Punkt2D(Punkt2D p) {  
14 |     this(p.x, p.y);  
15 | }
```

Instanziierung

Instanziierung in der Form

InstanziierungsAusdruck:

```
new KlassenBez ( [ Ausdruck { , Ausdruck } ] )
```

hat folgende Effekte:

- 1 **allokiert** (**Heap**-)Speicher für ein *KlassenBez*-Objekt (**OutOfMemoryError**, falls kein Speicherplatz mehr verfügbar)
- 2 **initialisiert Instanzvariablen** implizit/explicit
- 3 führt **Konstruktor mit passender Signatur** aus
- 4 liefert **Referenz** vom Typ *KlassenBez* auf das Objekt

(weitere Effekte später)

Spezialfall: **String-Konstante** instanziiert autom. **String-Objekt**
"Hallo" kann gelesen werden als `new String("Hallo")`

Methoden

Methoden

Methode beschreibt

- funktional: Zuordnung von Rückgabewert zu Argumenten (Typen gemäß Deklaration)
- nichtfunktional: Zustandsänderungen
- modellierend: Fähigkeit und Bedeutung
- technisch: Vorgehensweise (Folge von Anweisungen)

Deklaration

MethodenDeklaration:

MethodenKopf MethodenKoerper

MethodenKopf:

Zugriff [static] [...] TypOrVoid MethodenBez (ParamListe)

TypOrVoid:

Typ

void

MethodenKoerper:

{ { Anweisung } }

Beispiel:

```
8 | public static int max(int a, int b) {  
9 |     return a >= b ? a : b;  
10| }
```

Konventionen

Konventionen für die Deklaration von Methoden:

- in Klasse **erst Variablen**, dann **Methoden** deklarieren
- *Zugriff* normalerweise **public**
→ dann auch **Javadoc-Methodenkommentar** sinnvoll
- **private**: **Hilfsmethode** (intern ausgelagerte Aufgabe)
- *MethodenBez*: erst **Kleinbuchstabe**, dann **CamelCase**
Unterschied zu *VariablenBez*: Parameter-/Argumentliste (...)

Prinzip

Methodenkopf beschreibt „**Vertrag**“ mit **Anwender**:
„gibst Du mir dies (Argumente), gebe ich Dir das (Ergebnis)“
(außerdem: wer kann wie auf Methode **zugreifen**?)

Dokumentation beschreibt ggf. **Nebeneffekte** (side effects):

- Ein-/Ausgaben
- Zustandsänderungen (von Variablen)
- Reaktion bei Fehlern
- ...

Methodenkörper { ... } beschreibt, **wie** Aufgabe erfüllt wird
(nur für **Entwickler** relevant)

Design by Contract (DbC)

Design by Contract: Vertrag zwischen Anwender und Entwickler als zentrales Entwurfsmittel für Methoden:

- **Vorbedingung:** gültige Argumente, Zustand laut Doku.
- **Invarianten:** Einhaltung von Garantien bei der Ausführung (z.B. Zusammenhänge zwischen beteiligten Variablen)
- **Nachbedingung:** Resultat und Nebeneffekte/neuer Zustand (z.B.: geänderte Variablen, erfolgte Ein-/Ausgabe)

Aufruf

Methode kann **beliebig oft aufgerufen** werden:

MethodenAufrufAusdruck:

MethodenQualName (*[* *Ausdruck* *{*, *Ausdruck* *}* *]*)

MethodenQualName:

KlassenBez . *MethodenBez*

Referenz . *MethodenBez*

Ergebnis des Aufrufs ist **Wert** von *MethodenAufrufAusdruck*:

... Mathe.max(6 * 4 + 2, 4 * 7 - 3) ... **// Wert ist 26**

erst Auswertung der Argumente (von links nach rechts), dann Aufruf:

int i = 0;

... Mathe.max(i = 1, ++i) ... **// Wert ist 2**

Parameter

Wiederholung:

Parameter prinzipiell wie **lokale Variable**:

- hat **Typ** und **Name**, beschreibt **Speicherplatz**
- existiert nur für Dauer der Methodenausführung

aber: wird bei Aufruf mit **Wert des Arguments** initialisiert

- **Anzahl und Typen der Argumente** müssen zu **Parametern** passen („passen“ kann **Typumwandlung** involvieren – dazu später)

„**Parameter**“ auch: **formaler** (formal) **Parameter**

„**Argument**“ auch: **tatsächlicher** (actual) **Parameter**

Signatur

Signatur einer Methode: *MethodenBez* & *ParamTyp*-Liste
jedoch **ohne**: Zugriff, *static*, Rückgabetyt etc.

Anforderung: alle **Signaturen** in einer Klasse sind **verschieden**

Sinn: pro Aufruf darf nur eine Methode in Frage kommen

```
public static int max(int a, int b) { ... }
```

```
int x, y;
```

```
... Mathe.max(x, y) ...
```

Überladen

Methoden(namen) können innerhalb einer Klasse **überladen** werden:

- weitere Methode mit **selbem Namen**, **anderer Parameter(typ)liste**
- also: andere **Anzahl**, **Typen** und/oder **Reihenfolge** der Parameter
- Namen der Parameter egal, da nur in der Methode bekannt

Überladen bereits von **Konstruktoren** und **Operatoren** bekannt,
z.B. **+** für `int`, `double`, `String`

Überladen: Beispiele

bereits bekannt:

```
8 | public static int max(int a, int b) {  
9 |     return a >= b ? a : b;  
10| }
```

Überladen nach Argumentzahl:

```
18| public static int max(int a, int b, int c) {  
19|     return max(max(a, b), c);  
20| }
```

Überladen nach Argumenttyp:

```
27| public static double max(double a, double b) {  
28|     return a >= b ? a : b;  
29| }
```

Überladen: Gegenbeispiel

Anwendung: analoge Funktionalität für unterschiedliche Parameter

Fehlanwendung: unterschiedliche Funktionalität (Verwirrung!)

```
public static String max(int j) {  
    return "Max (" + j + " Jahre alt)";  
}
```

Lokale Variablen

Wiederholung:

lokale Variable

- wird in Methode (oder Konstruktor) deklariert

LokaleVariablenDeklaration:

Typ VariablenBez [= Ausdruck] ;

- existiert nur während der jeweiligen Methodenausführung
- modelliert Zwischenwerte der Methodenausführung
- nicht automatisch initialisiert: vor Verwendung Wert zuweisen!

lokale Variable (nur) sinnvoll, wenn der gespeicherte Wert

- später oder/und
- mehrfach (Spezialfall von „später“) verwendet wird

⇒ lok. Var. ist überflüssig, wenn nur einmal und sofort verwendet

return-Anweisung

Wiederholung:

Anweisung

```
return Ausdruck;
```

- kann an mehreren Stellen in einer Methode stehen
- beendet die Ausführung der Methode
- delegiert die Ausführung zurück zur Stelle des Aufrufs
- liefert den Wert von *Ausdruck* als Ergebnis des Aufrufs
(*Typ* muss dem **deklarierten Ergebnistyp entsprechen**)
- ist in Methode mit Ergebnistyp **erforderlich**

return-Anweisung in void-Methode

Wiederholung:

nur in Methode mit Ergebnistypplatzhalter **void** (kein Ergebnis)
oder in **Konstruktor**:

```
return;
```

Wirkung der Methode dann nur durch **Nebeneffekte**

return; am Ende von Methodenblock optional (überflüssig)
⇒ nur in **Fallunterscheidungen** zum vorzeitigen Beenden

Klassen- vs. Instanzmethoden

Klassen- vs. Instanzmethoden

für **Instanzmethoden** gilt, im Gegensatz zu **Klassenmethoden**:

- „kennen“ die **Instanz** (als vordef. Referenz **this** ~ „ich“)
- erlauben daher **Zugriff auf Elemente der Instanz**, also auf **Instanzvariablen** und **andere Instanzmethoden**
- können also nur **für Instanz aufgerufen** werden

für **Klassenmethoden** gilt, im Gegensatz zu **Instanzmethoden**:

- sind **instanzunabhängig**, können auch für Klasse aufgerufen werden: **X.m()** oder **x.m()**
- können **nur Klassenvariablen und -methoden** direkt verwenden

Konsequenz:

Klassenmethoden können **aus Instanzmethoden** aufgerufen werden, **Instanzmethoden** aber **nicht aus Klassenmethoden**.

Analogie Klassen-/Instanzmethode: Beispiel

```
1 | public class Mensch {  
    // ...  
5 | private Mensch p;      // Partner/in  
    // ...  
23 | public void heirat(Mensch m) {  
    // alternativ: heirat(this, m);  
24 |     this.p = m;  
25 |     m.p = this;  
26 | }  
  
    public static void heirat(Mensch t, Mensch m) {  
    // alternativ: t.heirat(m);  
        t.p = m;  
        m.p = t;  
    }  
    // ...  
33 | }
```


Analogie Klassen-/Instanzmethode

Achtung: hier nur Veranschaulichung, **kein korrektes Java!**

wie im Beispiel gezeigt, ließe sich Instanzmethode

```
public void heirat(Mensch m) { ... }
```

im Prinzip wie Klassenmethode

```
public static void heirat(Mensch this, Mensch m) { ... }
```

auffassen / übersetzen, entsprechend deren Aufruf

```
mo.heirat(ma)
```

wie

```
Mensch.heirat(mo, ma)
```

unter **Übergabe** der verwendeten **Referenz** als „Parameter“ **this**

aber: Zugriff auf Instanz immer über Instanzmethode ausführen

Klassen- vs. Instanzzugriff: Beispiel I

```
1 public class Bohrmaschine {
2     private static int a;           // Klassenvariable (KV)
3     private int nr;                 // Instanzvariable (IV)
4     public Bohrmaschine() {         // Konstruktor - wie IM
5         nr = ++a;                   // Zugriff KV(&KM) & IV(&IM)
6     }
7     public static int anzahl() {     // Klassenmethode (KM)
8         return a;                   // Zugriff KV(&KM)
9     }
10    public int nr() {                 // Instanzmethode (IM)
11        return nr;                  // Zugriff (KV&KM) & IV(&IM)
12    }
13    public String toString() {        // IM; Zugriff (KV&)KM & (IV&)IM
14        return "Bohrmaschine #" + nr() + "(von " + anzahl() + ")";
15    }
16 }
```

Klassen- vs. Instanzzugriff: Beispiel II

```
Bohrmaschine ba = new Bohrmaschine();  
Bohrmaschine bb = new Bohrmaschine();  
Bohrmaschine bc = new Bohrmaschine();  
Bohrmaschine bd = new Bohrmaschine();  
Bohrmaschine be = new Bohrmaschine();  
Bohrmaschine bf = new Bohrmaschine();  
Bohrmaschine bg = new Bohrmaschine();
```

```
String s = "" + bd.toString(); // "Bohrmaschine #4 (von 7)"
```

Anmerkung: Aufruf von toString nicht erforderlich;

```
String s = "" + bd;
```

ist äquivalent (dazu gleich)

Beispiel: Klasse Mensch (1)

```
1 public class Mensch {
2     private static int a; // Anzahl
3     private String n;     // Name
4     private int j;        // Geburtsjahr
5     private Mensch p;     // Partner/in
6     public Mensch(String n, int j) {
7         ++a;
8         this.n = n;
9         this.j = j;
10    }
11    public static int anzahl() {
12        return a;
13    }
14    public String name() {
15        return n;
16    }
```

Beispiel: Klasse Mensch (2)

```
17 public int geburtsjahr() {
18     return j;
19 }
20 public int alter(int j) {
21     return j - this.j;
22 }
23 public void heirat(Mensch m) {
24     this.p = m;
25     m.p = this;
26 }
27 public boolean equals(Mensch o) {
28     return this == o || j == o.j && n.equals(o.n);
29 }
30 public String toString() {
31     return n + " (" + j + ")";
32 }
33 }
```

Standard-Instanzmethode toString

es gibt „vorgegebene“ (später genauer) Standard-Instanzmethoden:

```
public String toString() { ... }
```

liefert Darstellung der Instanz als Zeichenkette

dient Typumwandlung von (nicht-null)-Referenz zu String:

```
mo + " & " + ma    // "Monika (1992) & Markus (1998)"  
mo + " & " + null  // "Monika (1992) & null"
```

Hinweis: toString wird „automatisch“ deklariert,
falls nicht bereitgestellt; liefert dann Zeichenkette wie:

```
"" + mo            // "Mensch@6bc7c054"
```

Bedeutung: „Mensch-Objekt mit Identifikationsnummer 6bc7c054“

Standard-Instanzmethode equals

```
public boolean equals(Klassentyp o) { ... }
```

vergleicht Instanz mit anderem Objekt gleichen Typs:

```
Mensch mo = new Mensch("Monika", 1992);  
Mensch ma = new Mensch("Markus", 1998);  
Mensch m2 = new Mensch("Monika", 1992);
```

```
... mo.equals(ma) ... // false  
... mo.equals(m2) ... // true  
... mo == m2 ... // false
```

- **equals** prüft **Gleichheit** („ist das gleiche wie“),
- **==** prüft **Identität** („ist das selbe wie“)

⇒ in der Regel **equals** verwenden, **==** nur in Sonderfall

Implementierung von equals

```
27 | public boolean equals(Mensch o) {  
28 |     return this == o || j == o.j && n.equals(o.n);  
29 | }
```

Identität (`this == o`) **Spezialfall** von Gleichheit, „billig“ zu prüfen

⇒ Prüfung vorweg (`||`) zur **eventuellen Optimierung**
(spart equals-Aufruf, wenn true – sonst aber Zusatzaufwand!)

int-Vergleich (`j == o.j`) billiger als für String/Objekt (`equals`)

⇒ Prüfung in dieser Reihenfolge (`&&`) zur **Optimierung**

Hinweis: `equals` wird „**automatisch**“ **deklariert**,
falls nicht bereitgestellt; liefert dann Wert von `this == o`

Beispiel Punkt2D (1)

```
1 public class Punkt2D {
2     private static int a;
3     private double x; // x-Koordinate
4     private double y; // y-Koordinate
5     public Punkt2D() {
6         ++a;
7     }
8     public Punkt2D(double x, double y) {
9         this();
10        this.x = x;
11        this.y = y;
12    }
13    public Punkt2D(Punkt2D p) {
14        this(p.x, p.y);
15    }
```

Beispiel Punkt2D (2)

```
16 public static int anzahl() {
17     return a;
18 }
19 public double x() {
20     return x;
21 }
22 public double y() {
23     return y;
24 }
25 public boolean equals(Punkt2D o) {
26     return this == o
27         || Mathe.equals(x, o.x) // Rundungstoleranz!
28         && Mathe.equals(y, o.y);
29 }
30 public String toString() {
31     return "(" + x + "," + y + ")";
32 }
```

Beispiel Punkt2D (3)

```
33 public Punkt2D verschiebe(double dx, double dy) {
34     x += dx;
35     y += dy;
36     return this;
37 }
38 public double abstand(Punkt2D p) {
39     double dx = x - p.x;
40     double dy = y - p.y;
41     return Math.sqrt(dx * dx + dy * dy); // Pythagoras
42 }
43 }
```

Rückgabe von `this` macht `verschiebe`-Aufruf zu Wertausdruck, dadurch Verwendung in anderen Ausdrücken möglich:

```
Punkt2D p = new Punkt2D(1, 2);
... abstand(p.verschiebe(3, 1), new Punkt2D()) ...
```

Übung: Datum

Aufgabenblatt 06, Aufgabe 01

Übung: Rational

Aufgabenblatt 06, Aufgabe 02

Übung: RechteckXYPerEcke

Aufgabenblatt 06, Aufgabe 03

Geteilter und exklusiver Zugriff

Referenzen als Parameter

durch **Methode** mit **veränderndem Zugriff** über **Referenz**:

```
public static void verdoppele(Punkt2D p) {  
    p.verschiebe(p.x(), p.y());  
}
```

wird direkt die **originale Instanz verändert**:

```
Punkt2D p = new Punkt2D(1, 2);  
verdoppele(p);  
// p.x() == 2  
// p.y() == 4
```


Nicht-Referenz-Typen als Parameter

im Kontrast dazu Methode mit nicht-Referenz-Parameter:

```
public static void verdoppele(int i) {  
    i *= 2;  
}
```

nach Aufruf ist das „Original“ unverändert:

```
int x = 1;  
verdoppele(x);  
// x == 1
```

da der Wert von x kopiert wird, wirkt die Änderung nur auf die Kopie

„call by value“ vs. „call by reference“

wichtig: in Java immer **Argumentübergabe per Kopie (call by value)**
Parameter von Referenztyp erlaubt aber **Änderung der Originalinstanz**

Hinweis: in **anderen Sprachen** ist dieser Mechanismus integriert,
referentielle Argumentübergabe (call by reference) ist dann auch
für z.B. primitive Typen möglich (**Referenzsemantik**)

Beispiel Kreis (1)

```
1 public class Kreis {
2     private Punkt2D m;
3     private double r;
4     public Kreis() {
5         this(0);
6     }
7     public Kreis(double r) {
8         this(new Punkt2D(), r);
9     }
10    public Kreis(Punkt2D m, double r) {
11        // ...
12    }
13    public Kreis(Kreis k) {
14        this(k.m, k.r);
15    }
16    public Punkt2D mitte() {
17        // ...
18    }
19 }
```

Teilen von Objekten

```
Punkt2D p = new Punkt2D(4, 1);  
Kreis k = new Kreis(p, 2);  
p.verschiebe(-1, 3);           // verschiebt sich k mit?
```

k verschöbe sich mit p, wenn der Konstruktor so aussähe:

```
public Kreis(Punkt2D m, double r) {  
    this.m = m; // 2. Referenz auf selbe Punkt2D-Instanz  
    this.r = r;  
}
```

falls Referenzen auf selbes Objekt in verschiedenen Objekten:
Änderungen durch einen Besitzer sind für alle wirksam
→ **Teilen (shared ownership)** des Objekts

Teilen vs. Kopieren

falls **Teilen nicht erwünscht** (Designentscheidung), **Kopieren** nötig:

```
10 public Kreis(Punkt2D m, double r) {  
11     this.m = new Punkt2D(m);  
12     this.r = r;  
13 }
```

Objekt hält eigene Instanz (**exclusive ownership**),
spätere **Änderung des Originals** ohne Folgen

gleicher Grund für **Rückgabe von Kopie** durch mitte():

```
17 public Punkt2D mitte() {  
18     return new Punkt2D(m);  
19 }
```

damit ist **unkritisch**:

```
k.mitte().verschiebe(2,1); // k wird nicht bewegt
```

Beispiel Kreis (2)

```
20 public double radius() {
21     return r;
22 }
23 public boolean istEnthalten(Punkt2D p) {
24     return m.abstand(p) <= r;
25 }
26 public boolean equals(Kreis o) {
27     return this==o || m.equals(o.m) && Mathe.equals(r, o.r);
28 }
29 public void verschiebe(double dx, double dy) {
30     m.verschiebe(dx, dy);
31 }
32 public String toString() {
33     return "[" + m + "," + r + "]";
34 }
35 }
```

Bindung

Begriffe

Bindung bedeutet Benennung eines Elements mit einem **Bezeichner**
z.B. Bindung eines Variablenbezeichners an einen Speicherplatz

Gültigkeitsbereich (**Scope**) einer **Bindung** ist der Bereich,
in dem die **Bindung besteht** (z.B. der **Block** der Deklaration)

Sichtbarkeitsbereich ist Teilbereich, in dem die **Bindung aktiv** ist

Kontrast: **Zugriffsrecht** (public etc.): **Zugriff erlaubt?**

Lebensdauer der Bindung währt **von Erzeugung bis Auflösung**,
bei Auflösung von Variablenbindung Freigabe des Speicherplatzes

Blockanweisung

Block muss nicht Methoden- oder Schleifenkörper sein; reiner Block

```
BlockAnweisung:  
{ { Anweisung } }
```

fungiert als Anweisung (später mehr dazu), erlaubt:

- weitere Strukturierung des Codes
- Begrenzung der Gültigkeit von Variablen:

```
...  
{                               // Zwischenabschnitt  
    int tmp = f(wert); // temporäre Hilfsvariable  
    aktion1(tmp);  
    aktion2(tmp);  
    ...  
}                               // 'tmp' ab hier ungültig  
...
```

Gültigkeitsbereich für lokale Variable

Scope für lokale Variable: ab Deklaration bis Ende des Blocks

```
{  
    int a = 1;      // ab hier 'a' gueltig  
    int b = 2;      // ab hier 'b' gueltig  
    {  
        int tmp = a; // ab hier 'tmp' gueltig  
        a = b;  
        b = tmp;  
    }              // ab hier 'tmp' ungueltig  
    // a == 2  
    // b == 1  
}                  // ab hier 'a', 'b' ungueltig
```

Übersicht

Bezeichner für	Gültigkeit	Lebensdauer
Klassenelement	Klassenkörper	i.W. Programmdauer
Instanzelement	Klassenkörper	Objektlebensdauer
Parameter	Methodenkörper	Methodenausführung
lokale Variable	ab Deklaration bis Ende Block	ab Deklarationsausführung bis Ende Blockausführung

Gültigkeit vs. Sichtbarkeit

in verschachtelten Gültigkeitsbereichen/Blöcken:

nur **jeweils innerste Bindung** aktiv/sichtbar, **äußere verdeckt**

werden Bezeichner von **Klassenelementen** in **Methode verdeckt**,
bleiben sie durch **qualifizierten Zugriff** erreichbar:

- **Instanzelemente** per **Selbstreferenz**:
`this.ElementBez...`
- **Klassenelemente** auch per **Klassenname**:
`KlassenBez.ElementBez...`

Beispiel

1./2. Spalte: **gültige** Werte für x/y; **sichtbarer** jeweils zuletzt

```

public class C {                                // 1      2
    public int f(int y) {                        // 1      2 # (Arg.wert)
        if (y > 0) {                            // 1      2 #
            int x = C.y;                        // 1 2     2 #
            x += this.x;                        // 1 3     2 #
            return x; // ergibt 3               // 1 3     2 #
        }                                       // 1      2 #
        return x; // ergibt 1                  // 1      2 #
    }                                           // 1      2
    private int x = 1;                          // 1      2
    private static int y = 2;                   // 1      2
}

```

Übung: Uhrzeit

Aufgabenblatt 06, Aufgabe 04

Methodenausführung

Speicherbereich eines Prozesses

Layout des Speicherbereichs eines Prozesses bei Ausführung:



- **Text**-Bereich: (statischer) auszuführender Programmcode
- **Data**-Bereich: statische (globale) Daten (Klassen)
- **Heap**-Bereich: dynamisch (mit `new`) erzeugte Daten (Objekte)
- **Stack**-Bereich: dynamisch pro Methodenaufruf erzeugte Daten (Parameter & lokale Variablen)

Fragen:

- welche Daten werden pro Methodenaufruf im Stack angelegt?
- wie wird der Stack verwaltet?

Speicherbedarf eines Methodenaufrufs

Methodenausführung benötigt Speicherplatz für:

- Parameter
- lokale Variablen
- Information, wohin (nach `return`) zurückzukehren ist
- ggf. weitere interne Hilfsdaten
- Größe des Speicherplatzes

dieser Speicherplatz soll

- **angelegt** werden, wenn die Methode aufgerufen wird
- **freigegeben** werden, wenn die Methode endet

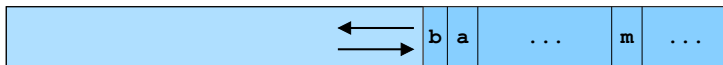
Lebensdauer des Speichers für Methodenaufrufe

wichtig: ruft Methode a Methode b auf, so bleibt Methode a aktiv, „wartet“ auf die Beendigung von b

- Speicher für b wird **angelegt** und dann wieder **freigegeben**, während Speicher für a **bestehen bleibt**
- **Lebensdauer** des Speichers für **aufgerufene Methode b** **enthalten in Lebensdauer** des Speichers der **aufrufenden Methode a**

Stack

natürliche Organisationsform bei beschriebener Verhaltensweise:
Stack (Stapel)



- bei Aufruf von b „wächst“ Stack um entspr. Speicherplatz (**Stack Frame**)
- bei Ende von b „schrumpft“ Stack um Speicherplatz
- Speicherplatz von a und anderen „wartenden“ Methoden bleibt
- insgesamt verfügbarer Speicherplatz ist begrenzt
→ ggf. **StackOverflowError**
(analog zu **OutOfMemoryError** bei Allokierung auf Heap)

main (m im Bild) wird aufgerufen von System-/JVM-Methoden